PixMatch 1.4 — Architecture

Measurement engine, application structure, verification and limits · version 1.4 · October 2026

PixMatch is a browser application for measuring fluid-mechanics image sequences (droplet height, contact angle, contact diameter, phase extent). It runs entirely client-side: images are decoded in the browser, measured in the browser, and exported from the browser. Nothing is uploaded.

This document explains how the code is organised, how a measurement travels from a pixel to a reported number, how every number is qualified (status, confidence, uncertainty), and where the limits are.

1. Two measurement paths

PixMatch has two ways to measure, deliberately kept separate:

Path	Where	What it is for
Interactive tools (legacy)	`src/lib/*`, `src/routes/index.tsx`	Draw a line, angle, ROI or wand region on a reference frame, capture a "recipe", replay it on every frame (tracked heights, diameters, contact angles, PIV). Flexible and manual.
Measurement pipeline (engine)	`src/scientific/*`, `src/components/pixmatch/PipelinePanel.tsx`	Fully specified, frame-independent, audited measurement of one phase's vertical extent. Reproducible and exportable with provenance.

Version 1.3 shipped the engine as a standalone folder that nothing in the app called. **Version 1.4 connects it**: the right-hand sidebar has a *Measurement pipeline (scientific engine)* card, the *Batch* menu has a shortcut to it, and the *Download* menu exports its results. The interactive tools are unchanged.

2. The pipeline

```
ImageData (RGBA, 8-bit)
  ▼
sRGB → XYZ (D65) → CIELAB                 color/rgbToLab.ts
  ▼
Exposure match (optional gain on L*a*b*)  segmentation/segmentFrame.ts
  ▼
CIEDE2000 distance → 2-D phase mask       color/ciede2000.ts, segmentation/phaseClassifier.ts
  ▼
ROI-safe morphology (open, close, fill)   segmentation/morphology.ts
  ▼
Connected components (area, largest)      segmentation/connectedComponents.ts
  ▼
Boundary extraction (edge, top/bottom)    geometry/boundary.ts
  ▼
Vertical bounds → pixel height            geometry/bounds.ts
  ▼
mm/pixel calibration → physical height    calibration/calibration.ts
  ▼
Confidence + uncertainty                  quality/confidence.ts, calibration/uncertainty.ts
  ▼
Validated MeasurementResult + status      execution/processFrame.ts
  ▼
Batch audit, CSV, JSON                    execution/processBatch.ts, export/export.ts
```

Every frame goes through this chain **independently**. No frame's result depends on another frame's pixels. The only cross-frame step is optional interpolation of missing frames (§6), which is off by default and always labelled.

2.1 Stage details

#	Stage	What happens	Key settings	Fails by
1	**Colour conversion**	8-bit sRGB is gamma-decoded with a 256-entry lookup table, multiplied by the sRGB→XYZ matrix (D65 white, Y = 1), then converted to CIELAB. Whole-image conversion allocates nothing per pixel.	–	Wrong colour space in the source image (see §11).
2	**Exposure match**	If enabled, L*, a* and b* are all multiplied by `targetMedianL / frameMedianL` (clamped 0.5–2). The target is the median L* of the reference frame.	on/off	Large exposure changes, or a scene whose median is not background.
3	**Phase classification**	CIEDE2000 distance from every pixel in the ROI to the phase colour. A pixel is in the phase when the distance ≤ tolerance. A cheap lower bound `abs(ΔL*) / 1.75` skips the full formula for pixels that are certainly too far (verified against the exact formula on 20 000 random pairs).	phase Lab, ΔE00 tolerance, ROI	Tolerance too tight (phase missing) or too loose (background leaks in).
4	**Morphology**	Opening removes specks, closing bridges small gaps, hole filling fills regions not connected to the image border. Square element, computed as two 1-D passes. Pixels outside the ROI are copied unchanged.	open px, close px, fill holes	Radius larger than the feature (erodes the phase away).
5	**Components**	4-connected labelling. Components below the minimum area are dropped; "largest only" keeps a single region.	min area, largest only	Phase legitimately split in two (e.g. a satellite droplet) when "largest only" is on.
6	**Boundary extraction**	Edge pixels = phase pixels with a background 4-neighbour (or on the image border). Also records each column's topmost and bottommost phase row.	–	–
7	**Vertical bounds**	Top = highest top edge, bottom = lowest bottom edge. With *trim*, the outer N % of phase pixels at each end is ignored so isolated specks cannot move an edge.	convention, trim %, baseline row	Stray pixels when no component filter is used.
8	**Pixel height**	`max − min` (pixel-centre) or `max − min + 1` (inclusive). With a baseline row: `baseline − top` (+1 if inclusive).	convention, baseline	Baseline above the top edge → height 0, status LOW_CONFIDENCE.
9	**Calibration**	`mm = pixels × mm/px`. A missing or non-positive scale is rejected before any frame is processed.	mm/px, ±σ	–
10	**Confidence & uncertainty**	See §4 and §5.	min confidence	–

3. Data model

Defined in `src/scientific/types.ts`.

Input. `ImageFrame { frameIndex, name?, width, height, data: Uint8ClampedArray }` — exactly what `getImageData` returns.

Recipe. `BatchRecipe` holds the calibration, one or more `MeasurementDefinition`s (phase colour, tolerance, ROI, morphology, component filter, exposure match, percentile trim, convention, baseline), the confidence threshold and the interpolation switch. `buildHeightRecipe()` (`execution/recipeBuilder.ts`) turns the handful of UI settings into a full recipe. Only `HEIGHT` is implemented; any other quantity is rejected up front.

Output. One `MeasurementResult` per frame per measurement:

Field	Meaning
`frameIndex`, `frameName`	Which image.
`minY`, `maxY`, `centroidY`	Top edge row, bottom edge row, mean row of the phase. `null` when no phase was found.
`pixelHeight`, `physicalHeightMm`	The measurement. `null` — never a guess — when it could not be made.
`status`	`VALID`, `LOW_CONFIDENCE`, `MISSING_PHASE`, `INVALID_FRAME`, `SEGMENTATION_FAILED`, `INTERPOLATED`.
`confidence`, `confidenceParts`	0–1 score and its eight terms.
`uncertainty`	Calibration, segmentation and combined 1-σ in mm.
`areaPixels`, `componentCount`, `meanDeltaE`, `boundaryContinuity`	Measured quality diagnostics.
`baselineY`, `notes`	Baseline used; human-readable reasons (e.g. `baseline_not_below_top`, `illumination_gain=1.234`, `interpolated_between_frames_0_and_2`).
`calibrationScaleMmPerPixel`, `algorithmVersion`, `measurementConvention`	Provenance.

Status rules. A frame is `INVALID_FRAME` when its pixel buffer is inconsistent, `MISSING_PHASE` when no pixel is within tolerance, `SEGMENTATION_FAILED` when processing threw (the message is kept in `notes`), `LOW_CONFIDENCE` when confidence is below the threshold (the number is still reported but flagged), and `VALID` otherwise. A bad frame never stops the batch.

4. Confidence

Confidence is the weighted mean of eight terms, each between 0 and 1 and each **computed from the frame** (none is a fixed constant). Default weights sum to 1.

Term (weight)	Definition	Low when
area (0.20)	`min(1, phase pixels / 500)`	the phase is tiny
continuity (0.30)	share of boundary pixels that touch ≥ 2 other boundary pixels	the outline is broken or dusty
compactness (0.10)	phase area ÷ bounding-box area	the shape is thin or scattered
colorMargin (0.15)	`1 − meanΔE00 / tolerance` over phase pixels	pixels barely pass the tolerance
components (0.10)	largest component ÷ total phase area	the phase is fragmented
unknown (0.05)	`1 − ambiguous / (ambiguous + phase)`, ambiguous = pixels with tolerance < ΔE00 ≤ 1.5 × tolerance	many pixels are "almost" the phase
contrast (0.05)	`ΔE00(mean inside, mean outside) / (3 × tolerance)`, capped at 1	phase and background look alike
smoothness (0.05)	share of neighbouring columns whose top edges differ by ≤ 3 px	the top edge is ragged

Confidence is a **quality score, not a probability**. It ranks frames and flags suspicious ones; the 0.6 default threshold is a starting point you should tune for your images.

5. Uncertainty and calibration

- **Segmentation uncertainty** = ½ pixel × mm/px (an edge is only known to the nearest pixel).
- **Calibration uncertainty** = σ(mm/px) × pixel height. σ is entered by the user; if left blank it is 0 and the combined value is the segmentation term only.
- **Combined** = square root of the sum of squares (independent contributions).

The scale comes from the needle-gauge / known-distance calibration (`pixels per unit` + unit). `mmPerPixelFromScale()` converts mm, µm, cm, m and in to mm/px; `px` or an uncalibrated scale gives `null`, and the pipeline then refuses to run until a scale is typed in. One mm/px is applied to every frame; frames whose size differs from the first frame produce an audit warning.

---

6. Batch, audit and interpolation

`BatchSession` (`execution/processBatch.ts`) accepts frames one at a time, so the page never needs a second copy of the whole sequence in memory. `finish()` returns:

- all `MeasurementResult`s;
- a `ProcessingAudit`: experiment id, algorithm version (`pixmatch-engine-1.4.0`), calibration, segmentation settings, all measurement definitions, frame names, status counts, warnings, start/end timestamps.

Interpolation is off unless the user ticks *Fill gaps by interpolation*. It only fills a `MISSING_PHASE` frame lying *between* two `VALID` frames of the same measurement (linear in frame index), never extrapolates, marks the result `INTERPOLATED`, sets its confidence to the lower of its two neighbours, and writes a note naming the frames used.

Exports (`export/export.ts`): CSV (one row per frame, includes status, all uncertainty terms, all eight confidence terms, notes), JSON (results + audit), and the audit record alone.

---

7. Threading

`worker/engine.worker.ts` runs the engine off the UI thread. `worker/client.ts` creates a runner:

1. `createRunner()` validates the recipe, starts the worker and waits for a `READY` handshake (8 s timeout).
2. If the browser has no `Worker`, or the worker fails to start, the **same engine runs on the main thread**, yielding between frames. The panel reports which mode ran.
3. The panel sends one frame at a time: `FRAME` (pixel buffer transferred, not copied) → `FRAME_RESULT`. At the end, `FINISH` → `RESULT`.
4. *Cancel* stops after the current frame and discards the run.

The client is imported with a dynamic `import()` from the panel's *Run* handler, so server rendering never touches `Worker`.

---

8. User interface

`PipelinePanel.tsx` is a self-contained React component. The app (`routes/index.tsx`) passes it: the frame list, a function that returns a frame's decoded `ImageData` (the app's existing cache), the calibrated scale, the last Point-probe colour, the last Wand-region mean colour, the selected line (for a baseline) and callbacks. The panel has no access to any other app state.

Workflow:

1. **Phase colour** — probe a pixel of the phase with the Point tool, or trace it with the Wand, then press *Use probed colour* / *Use wand region* (or type L*a*b*). Set the ΔE00 tolerance.
2. **Mask cleanup** — opening, closing, hole filling, minimum area, largest-region-only, outlier trim, optional exposure match.
3. **Height & calibration** — pixel convention, optional baseline row (can be taken from the selected line), mm/px (from calibration, or typed), scale σ, low-confidence threshold, interpolation switch.

4. **Live preview** — the same `processFrame` code on one frame: red mask, yellow extracted boundary, cyan top/bottom rows, status and numbers. Tune the settings here before running the batch.
5. **Run** — progress, cancel, results table (click a row to jump to that frame), height-versus-frame chart, warnings, CSV / JSON / audit buttons. If settings change after a run, the table is flagged as stale.
6. **Self-check** — runs 17 known-answer checks in the browser (§10).

The *Download* menu also offers the latest pipeline results as CSV, JSON and an audit record.

---

9. Code map

```
src/
  routes/index.tsx            app shell; mounts PipelinePanel, adds menu and download items
  components/pixmatch/
    PipelinePanel.tsx         engine UI (§8)
    CalibrationPanel.tsx      needle-gauge / known-distance scale
    DetectPanel.tsx           interactive CIELAB detection (legacy path)
    ResultsTable.tsx, HistogramView.tsx
  lib/                        interactive tools, PIV, exports, storage (legacy path)
  scientific/                 THE ENGINE (pure TypeScript, no DOM, no React)
    types.ts                  all shared types
    color/                    rgbToLab.ts (fast LUT conversion, labToRgb), ciede2000.ts
    segmentation/             phaseClassifier, morphology, connectedComponents, segmentFrame
    geometry/                 boundary.ts, bounds.ts, metrics.ts
    calibration/              calibration.ts (mm/px), uncertainty.ts
    quality/confidence.ts     the eight confidence terms
    execution/                processFrame, processBatch (BatchSession), recipeBuilder
    export/export.ts          CSV / JSON / audit
    audit/audit.ts            audit validation
    worker/                   engine.worker.ts, client.ts
    validation/referenceTests.ts   in-browser self-check + synthetic frames
    index.ts                  public exports (worker client excluded on purpose)
tests/engine.test.ts          37 Node tests (run with: npx tsx tests/engine.test.ts)
docs/scientific-engine-v4/    earlier design notes (superseded by this file)
```

The engine has no dependency on React, the DOM or any package, so it can be unit-tested in Node and reused elsewhere.

---

10. Verification

What was run for this release:

- **37 Node tests** (`tests/engine.test.ts`): colour conversion against known values (white, black, red; round trip), CIEDE2000 against published reference pairs (Sharma, Wu & Dalal 2005), the early-exit bound against the exact formula on 20 000 random pairs, morphology and component behaviour on hand-built masks, boundary extraction, shrinking-droplet sequence recovered frame by frame, missing / invalid frames, interpolation rules, exposure matching on a darkened frame, ROI, size-mismatch warning, recipe validation, CSV/audit export, unit conversion, and a speed check (a 1600 × 1200 frame segments in about half a second in Node).
- **In-browser self-check** (17 checks, button in the panel).
- **Headless Chromium run** of the real `PipelinePanel` with synthetic frames, in both worker mode and forced main-thread fallback: probe → preview → run → table → CSV download, with identical results (heights 100, 90, …, 50 px → 5.0 … 2.5 mm; a dust speck correctly ignored) and no console errors.
- TypeScript strict type-check of the engine; the panel was type-checked against stand-in React types.

What was **not** run: a full `vite build` of the whole app and a Vercel deployment (the build environment had no network to install dependencies). Deploy and check the build log.

---

11. Assumptions and limits

- **Colour space.** Input is assumed to be sRGB with a D65 white point. Images in another profile (Adobe RGB, camera RAW conversions) are not colour-managed.
- **Single scale.** One mm/px is applied to every frame. Zoom or focus changes between frames are not corrected.
- **Exposure match** is a global gain; it handles small uniform drifts, not uneven lighting or large changes.
- **Height only.** Span / diameter, contact angle and area are not part of the engine pipeline yet (the interactive tools still provide them). The hooks exist (`extractHorizontalBounds`, the `quantity` field), but unsupported quantities are rejected rather than silently ignored.
- **Pixel-level edges.** Edges are located to the nearest pixel; there is no sub-pixel refinement. The ½-pixel segmentation uncertainty reflects this.
- **Uncertainty** covers calibration and edge quantisation only. It does not include the effect of the ΔE tolerance choice; vary the tolerance to see that sensitivity.
- **Largest-region-only** assumes one phase body per frame.
- **Confidence** is a heuristic quality score with fixed default weights, not a calibrated probability.
- **Interactive vs engine numbers.** The legacy tools and the engine are separate implementations; their results can differ by a few pixels because they define edges differently (tracked lines vs. mask extremes).
- **Performance.** Cost grows with frame area and (for morphology) with the radius. Large frames are processed off the UI thread, but memory use is dominated by one decoded frame at a time.

---

12. Relationship to the "Extended Scientific Image Analysis" logic

The supplied 14-step logic maps onto the engine as follows:

Step in the supplied logic	Where it is implemented
ImageData	`ImageFrame`
RGB → XYZ / CIELAB	`color/rgbToLab.ts`
CIEDE2000 phase segmentation	`color/ciede2000.ts`, `segmentation/phaseClassifier.ts`
ROI-safe morphology	`segmentation/morphology.ts` (pixels outside the ROI are copied unchanged)
Connected components	`segmentation/connectedComponents.ts`
Actual boundary extraction	`geometry/boundary.ts` (added in 1.4; earlier the bounds were read straight from mask rows)
Pixel measurement	`geometry/bounds.ts`
mm/pixel calibration → physical measurement	`calibration/calibration.ts`, `processFrame.ts`
Confidence	`quality/confidence.ts`
Uncertainty	`calibration/uncertainty.ts`
Validated MeasurementResult	`processFrame.ts` + status rules (§3)
Batch audit / CSV / JSON	`processBatch.ts`, `export/export.ts`

The sequence is a sound specification: it puts cleanup *before* boundary extraction (so specks and holes cannot move the edge), keeps calibration out of the image stages, and ends in an audited, qualified result. Its main practical value in this project was making "actual boundary extraction" an explicit stage; the rest was already present in the 1.3 engine but not connected to the app.

---

13. Deployment notes

- Build: `vite build` (TanStack Start + Nitro; Nitro selects the Vercel preset on Vercel).
- Keep the **folder structure** when pushing to GitHub. Dragging loose files into GitHub's web uploader flattens folders and breaks imports; drag the folders, or use git.
- `package.json` pins `@tanstack/react-start` 1.168.60 and `@tanstack/react-router` 1.170.41, and the `overrides` block forces `@tanstack/start-server-core` 1.169.39 so Vercel's security block (CVE-2026-102989) does not trip on a transitive dependency.
- No lockfile is shipped. Vercel then installs with npm. Run `bun install` locally and commit `bun.lock` if you want pinned, reproducible installs.

---

14. Extending the engine

- **New quantity** (e.g. span): add a function beside `extractVerticalBounds`, extend `processFrame` for that `quantity`, add the field(s) to `MeasurementResult`, add it to `CSV_HEADER`, and add a known-answer test. Remove it from the "unsupported" check in `validateRecipe`.
- **New confidence term:** add it to `ConfidenceParts`, `DEFAULT_CONFIDENCE_WEIGHTS` and `scoreConfidence`; keep it measured, never constant.
- **Always:** add a synthetic fixture to `tests/engine.test.ts` with an answer you can compute by hand, and bump `ENGINE_VERSION` in `recipeBuilder.ts` so exported results identify the code that produced them.

---

15. Changes from 1.3 to 1.4

- Engine **connected to the app** (pipeline card, menu shortcut, downloads).
- Engine corrected: confidence terms are now measured (previously five were fixed constants); component count is measured (was hard-coded to 1); calibration is validated before processing (an invalid scale was previously reported as a segmentation failure); several settings that were accepted but ignored are now implemented (exposure match, percentile trim, baseline row, largest-region filter) or removed.
- New stage: **boundary extraction**; continuity and smoothness now use the real outline.
- Faster: lookup-table colour conversion, early-exit ΔE, separable morphology, array-free component labelling.
- Frames stream to the worker one at a time; automatic main-thread fallback; cancel support.
- Audit record extended (definitions, frame names, status counts, warnings); CSV extended (frame name, all uncertainty and confidence terms, notes).
- In-browser self-check, 37 Node tests, version string `pixmatch-engine-1.4.0`.